

Event Management System — Source Code

```
import tkinter as tk
from tkinter import ttk, messagebox
from openpyxl import Workbook, load_workbook
import os

# =====
# EVENT MANAGEMENT SYSTEM
# Python + Tkinter + Excel
# =====

FILE_NAME = "events.xlsx"

HEADERS = [
    "Event ID",
    "Event Name",
    "Event Date",
    "Event Time",
    "Venue",
    "Organizer",
    "Contact",
    "Event Type"
]

# =====
# EXCEL FUNCTIONS
# =====

def create_excel():
    """Create Excel file if it does not exist."""

    if not os.path.exists(FILE_NAME):

        workbook = Workbook()

        worksheet = workbook.active
        worksheet.title = "Events"

        worksheet.append(HEADERS)

        workbook.save(FILE_NAME)

def get_all_events():
    """Read all event records from Excel."""

    create_excel()

    workbook = load_workbook(FILE_NAME)
    worksheet = workbook["Events"]

    data = []

    for row in worksheet.iter_rows(
        min_row=2,
        values_only=True
    ):

        if any(value is not None for value in row):
            data.append(list(row))

    workbook.close()

    return data

def add_event_to_excel(values):
    """Add a new event to Excel."""

    create_excel()

    workbook = load_workbook(FILE_NAME)

    worksheet = workbook["Events"]

    worksheet.append(values)

    workbook.save(FILE_NAME)

    workbook.close()

def update_event_in_excel(event_id, values):
    """Update an existing event."""

    create_excel()

    workbook = load_workbook(FILE_NAME)

    worksheet = workbook["Events"]

    for row in range(2, worksheet.max_row + 1):

        if str(worksheet.cell(row, 1).value) == str(event_id):

            for column, value in enumerate(values, start=1):
```

```

        worksheet.cell(
            row,
            column
        ).value = value

        workbook.save(FILE_NAME)

        workbook.close()

        return True

workbook.close()

return False

def delete_event_from_excel(event_id):
    """Delete an event from Excel."""

    create_excel()

    workbook = load_workbook(FILE_NAME)

    worksheet = workbook["Events"]

    for row in range(2, worksheet.max_row + 1):

        if str(worksheet.cell(row, 1).value) == str(event_id):

            worksheet.delete_rows(row, 1)

            workbook.save(FILE_NAME)

            workbook.close()

            return True

    workbook.close()

    return False

# =====
# MAIN APPLICATION
# =====

class EventManagementSystem:

    def __init__(self, root):

        self.root = root

        self.root.title(
            "Event Management System"
        )

        self.root.geometry(
            "1050x650"
        )

        self.root.resizable(
            False,
            False
        )

        create_excel()

        self.login_page()

# =====
# CLEAR WINDOW
# =====

def clear_window(self):

    for widget in self.root.winfo_children():

        widget.destroy()

# =====
# LOGIN PAGE
# =====

def login_page(self):

    self.clear_window()

    frame = tk.Frame(
        self.root,
        padx=40,
        pady=30
    )

    frame.pack(
        expand=True
    )

    tk.Label(
        frame,

```

```

        text="Event Management System",
        font=("Arial", 24, "bold")
    ).grid(
        row=0,
        column=0,
        columnspan=2,
        pady=20
    )

    tk.Label(
        frame,
        text="Username",
        font=("Arial", 12)
    ).grid(
        row=1,
        column=0,
        sticky="w",
        pady=10
    )

    self.username_entry = tk.Entry(
        frame,
        width=30,
        font=("Arial", 12)
    )

    self.username_entry.grid(
        row=1,
        column=1,
        pady=10
    )

    tk.Label(
        frame,
        text="Password",
        font=("Arial", 12)
    ).grid(
        row=2,
        column=0,
        sticky="w",
        pady=10
    )

    self.password_entry = tk.Entry(
        frame,
        width=30,
        show="*",
        font=("Arial", 12)
    )

    self.password_entry.grid(
        row=2,
        column=1,
        pady=10
    )

    tk.Button(
        frame,
        text="Login",
        width=20,
        command=self.login,
        font=("Arial", 12, "bold")
    ).grid(
        row=3,
        column=0,
        columnspan=2,
        pady=20
    )

    tk.Label(
        frame,
        text="Demo Login: admin / 1234",
        font=("Arial", 10)
    ).grid(
        row=4,
        column=0,
        columnspan=2
    )

# =====
# LOGIN VALIDATION
# =====

def login(self):

    username = self.username_entry.get().strip()

    password = self.password_entry.get().strip()

    if username == "admin" and password == "1234":

        self.dashboard()

    else:

        messagebox.showerror(
            "Login Failed",
            "Invalid username or password."
        )

# =====

```

```

# DASHBOARD
# =====

def dashboard(self):

    self.clear_window()

    top = tk.Frame(
        self.root,
        bg="#1f4e78",
        height=80
    )

    top.pack(
        fill="x"
    )

    tk.Label(
        top,
        text="Event Management System",
        bg="#1f4e78",
        fg="white",
        font=("Arial", 24, "bold")
    ).pack(
        side="left",
        padx=25,
        pady=20
    )

    tk.Button(
        top,
        text="Logout",
        command=self.login_page,
        width=10
    ).pack(
        side="right",
        padx=20
    )

    menu = tk.Frame(
        self.root,
        pady=20
    )

    menu.pack()

    buttons = [
        ("Add Event", self.add_event_page),
        ("View Events", self.view_events_page),
        ("Search Event", self.search_event_page),
        ("Update Event", self.update_event_page),
        ("Delete Event", self.delete_event_page),
        ("Clear / Reset", self.add_event_page)
    ]

    for i, (text, command) in enumerate(buttons):

        tk.Button(
            menu,
            text=text,
            width=18,
            height=2,
            command=command,
            font=("Arial", 11, "bold")
        ).grid(
            row=i // 3,
            column=i % 3,
            padx=12,
            pady=12
        )

    tk.Label(
        self.root,
        text="Manage events easily using Python, Tkinter and Excel",
        font=("Arial", 15)
    ).pack(
        pady=40
    )

# =====
# ADD EVENT PAGE
# =====

def add_event_page(self):

    self.clear_window()

    tk.Label(
        self.root,
        text="Add Event",
        font=("Arial", 22, "bold")
    ).pack(
        pady=15
    )

    form = tk.Frame(
        self.root
    )

    form.pack()

```

```

self.form_entries = {}

fields = [
    "Event ID",
    "Event Name",
    "Event Date",
    "Event Time",
    "Venue",
    "Organizer",
    "Contact",
    "Event Type"
]

for i, field in enumerate(fields):

    tk.Label(
        form,
        text=field,
        font=("Arial", 11)
    ).grid(
        row=i,
        column=0,
        sticky="w",
        padx=10,
        pady=6
    )

    entry = tk.Entry(
        form,
        width=40,
        font=("Arial", 11)
    )

    entry.grid(
        row=i,
        column=1,
        padx=10,
        pady=6
    )

    self.form_entries[field] = entry

tk.Button(
    self.root,
    text="Save Event",
    width=18,
    command=self.save_event,
    font=("Arial", 11, "bold")
).pack(
    pady=10
)

tk.Button(
    self.root,
    text="Back to Dashboard",
    width=18,
    command=self.dashboard
).pack()

def get_form_values(self):

    return [
        self.form_entries[field]
        .get()
        .strip()
        for field in HEADERS
    ]

def save_event(self):

    values = self.get_form_values()

    if any(value == "" for value in values):

        messagebox.showwarning(
            "Missing Data",
            "Please fill all fields."
        )

        return

    existing_ids = [
        str(row[0])
        for row in get_all_events()
    ]

    if values[0] in existing_ids:

        messagebox.showerror(
            "Duplicate ID",
            "Event ID already exists."
        )

        return

    add_event_to_excel(values)

    messagebox.showinfo(
        "Success",

```

```

        "Event added successfully."
    )

    self.clear_form()

def clear_form(self):
    for entry in self.form_entries.values():
        entry.delete(
            0,
            tk.END
        )

# =====
# TREEVIEW
# =====

def create_treeview(self, parent):
    columns = HEADERS

    tree = ttk.Treeview(
        parent,
        columns=columns,
        show="headings",
        height=18
    )

    widths = [
        80, 150, 100, 90,
        130, 130, 110, 120
    ]

    for col, width in zip(columns, widths):
        tree.heading(
            col,
            text=col
        )

        tree.column(
            col,
            width=width,
            anchor="center"
        )

    scrollbar_y = ttk.Scrollbar(
        parent,
        orient="vertical",
        command=tree.yview
    )

    scrollbar_x = ttk.Scrollbar(
        parent,
        orient="horizontal",
        command=tree.xview
    )

    tree.configure(
        yscrollcommand=scrollbar_y.set,
        xscrollcommand=scrollbar_x.set
    )

    tree.grid(
        row=0,
        column=0,
        sticky="nsew"
    )

    scrollbar_y.grid(
        row=0,
        column=1,
        sticky="ns"
    )

    scrollbar_x.grid(
        row=1,
        column=0,
        sticky="ew"
    )

    return tree

def fill_tree(self, tree, data):
    for item in tree.get_children():
        tree.delete(item)

    for row in data:
        tree.insert(
            "",
            tk.END,
            values=row
        )

# =====

```

```

# VIEW EVENTS
# =====

def view_events_page(self):

    self.clear_window()

    tk.Label(
        self.root,
        text="View Events",
        font=("Arial", 22, "bold")
    ).pack(
        pady=15
    )

    table_frame = tk.Frame(
        self.root
    )

    table_frame.pack(
        fill="both",
        expand=True,
        padx=10,
        pady=10
    )

    tree = self.create_treeview(
        table_frame
    )

    self.fill_tree(
        tree,
        get_all_events()
    )

    tk.Button(
        self.root,
        text="Back to Dashboard",
        command=self.dashboard,
        width=20
    ).pack(
        pady=10
    )

# =====
# SEARCH EVENT
# =====

def search_event_page(self):

    self.clear_window()

    tk.Label(
        self.root,
        text="Search Event",
        font=("Arial", 22, "bold")
    ).pack(
        pady=15
    )

    search_frame = tk.Frame(
        self.root
    )

    search_frame.pack(
        pady=5
    )

    tk.Label(
        search_frame,
        text="Event ID or Event Name:",
        font=("Arial", 11)
    ).grid(
        row=0,
        column=0,
        padx=5
    )

    self.search_entry = tk.Entry(
        search_frame,
        width=30
    )

    self.search_entry.grid(
        row=0,
        column=1,
        padx=5
    )

    tk.Button(
        search_frame,
        text="Search",
        command=self.search_event
    ).grid(
        row=0,
        column=2,
        padx=5
    )

    table_frame = tk.Frame(

```

```

        self.root
    )

    table_frame.pack(
        fill="both",
        expand=True,
        padx=10,
        pady=10
    )

    self.search_tree = self.create_treeview(
        table_frame
    )

    self.fill_tree(
        self.search_tree,
        get_all_events()
    )

    tk.Button(
        self.root,
        text="Back to Dashboard",
        command=self.dashboard,
        width=20
    ).pack(
        pady=10
    )

def search_event(self):

    keyword = self.search_entry.get().strip().lower()

    if keyword == "":

        self.fill_tree(
            self.search_tree,
            get_all_events()
        )

        return

    results = []

    for row in get_all_events():

        event_id = str(row[0]).lower()
        event_name = str(row[1]).lower()

        if keyword in event_id or keyword in event_name:

            results.append(row)

    self.fill_tree(
        self.search_tree,
        results
    )

    if not results:

        messagebox.showinfo(
            "Search",
            "No event found."
        )

# =====
# UPDATE EVENT
# =====

def update_event_page(self):

    self.clear_window()

    tk.Label(
        self.root,
        text="Update Event",
        font=("Arial", 22, "bold")
    ).pack(
        pady=15
    )

    top = tk.Frame(
        self.root
    )

    top.pack(
        pady=5
    )

    tk.Label(
        top,
        text="Enter Event ID:"
    ).grid(
        row=0,
        column=0,
        padx=5
    )

    self.update_id_entry = tk.Entry(
        top,

```

```

        width=25
    )

    self.update_id_entry.grid(
        row=0,
        column=1,
        padx=5
    )

    tk.Button(
        top,
        text="Load Event",
        command=self.load_event_for_update
    ).grid(
        row=0,
        column=2,
        padx=5
    )

    form = tk.Frame(
        self.root
    )

    form.pack(
        pady=10
    )

    self.update_entries = {}

    for i, field in enumerate(HEADERS):

        tk.Label(
            form,
            text=field
        ).grid(
            row=i,
            column=0,
            sticky="w",
            padx=10,
            pady=5
        )

        entry = tk.Entry(
            form,
            width=40
        )

        entry.grid(
            row=i,
            column=1,
            padx=10,
            pady=5
        )

        self.update_entries[field] = entry

    tk.Button(
        self.root,
        text="Update Event",
        command=self.update_event,
        width=18
    ).pack(
        pady=8
    )

    tk.Button(
        self.root,
        text="Back to Dashboard",
        command=self.dashboard,
        width=18
    ).pack()

def load_event_for_update(self):

    event_id = self.update_id_entry.get().strip()

    for row in get_all_events():

        if str(row[0]) == event_id:

            for field, value in zip(HEADERS, row):

                self.update_entries[field].delete(
                    0,
                    tk.END
                )

                self.update_entries[field].insert(
                    0,
                    "" if value is None else str(value)
                )

            return

    messagebox.showerror(
        "Not Found",
        "Event ID not found."
    )

def update_event(self):
    values = [

```

```

        self.update_entries[field]
        .get()
        .strip()
        for field in HEADERS
    ]

    if any(value == "" for value in values):

        messagebox.showwarning(
            "Missing Data",
            "Please fill all fields."
        )

        return

    old_id = self.update_id_entry.get().strip()

    if old_id == "":

        messagebox.showwarning(
            "Missing ID",
            "Enter Event ID first."
        )

        return

    if update_event_in_excel(old_id, values):

        messagebox.showinfo(
            "Success",
            "Event updated successfully."
        )

        self.dashboard()

    else:

        messagebox.showerror(
            "Error",
            "Event ID not found."
        )

# =====
# DELETE EVENT
# =====

def delete_event_page(self):

    self.clear_window()

    tk.Label(
        self.root,
        text="Delete Event",
        font=("Arial", 22, "bold")
    ).pack(
        pady=20
    )

    tk.Label(
        self.root,
        text="Enter Event ID:",
        font=("Arial", 12)
    ).pack(
        pady=5
    )

    self.delete_id_entry = tk.Entry(
        self.root,
        width=30,
        font=("Arial", 12)
    )

    self.delete_id_entry.pack(
        pady=5
    )

    tk.Button(
        self.root,
        text="Delete Event",
        command=self.delete_event,
        width=18,
        font=("Arial", 11, "bold")
    ).pack(
        pady=15
    )

    tk.Button(
        self.root,
        text="Back to Dashboard",
        command=self.dashboard,
        width=18
    ).pack()

def delete_event(self):

    event_id = self.delete_id_entry.get().strip()

    if event_id == "":
        messagebox.showwarning(

```

```

        "Missing ID",
        "Enter Event ID."
    )

    return

found = any(
    str(row[0]) == event_id
    for row in get_all_events()
)

if not found:

    messagebox.showerror(
        "Not Found",
        "Event ID not found."
    )

    return

answer = messagebox.askyesno(
    "Confirm Delete",
    "Are you sure you want to delete this event?"
)

if answer:

    if delete_event_from_excel(event_id):

        messagebox.showinfo(
            "Deleted",
            "Event deleted successfully."
        )

        self.delete_id_entry.delete(
            0,
            tk.END
        )

# =====
# RUN APPLICATION
# =====

if __name__ == "__main__":

    root = tk.Tk()

    app = EventManagementSystem(root)

    root.mainloop()

```